

RAPPORT D'AUDIT · SÉCURITÉ APPLICATIVE

Audit de sécurité OWASP WSTG

Évaluation de la sécurité de l'application ACRA selon l'OWASP Web
Security Testing Guide v4.2, en vue de la publication de la version 1.0.

Application ACRA — plateforme d'analyse de risques EBIOS RM (Next.js 16 / Prisma / PostgreSQL)

Périmètre Code source, authentification, autorisation, gestion de session, stockage des secrets, en-têtes HTTP

Référentiel OWASP WSTG v4.2 · OWASP Top 10 2021 · ANSSI / ISO 27001

Date 14 juin 2026 · **Version** 1.0 · **Pré-publication**

1. Synthèse exécutive

ACRA présente une **posture de sécurité solide et mature** pour une application en pré-version 1.0. Les fondamentaux d'authentification, de gestion de session, de chiffrement des secrets et de contrôle d'accès sont correctement implémentés et couverts par des tests automatisés. Aucune vulnérabilité critique ou élevée n'a été identifiée sur les dépendances de production. Quelques durcissements de défense en profondeur sont recommandés avant une exposition Internet à fort trafic.

A-

NOTE GLOBALE

0

CRITIQUE / ÉLEVÉ

4

MOYEN

3

FAIBLE / INFO

Domaine	Verdict	Commentaire
Authentification (ATHN)	Conforme	bcrypt (coût 12), anti-bruteforce, verrouillage, expiration de mot de passe
Gestion de session (SESS)	Conforme	JWT next-auth, cookie httpOnly/SameSite, durée 8 h, secure en HTTPS
Autorisation (ATHZ)	Conforme	RBAC centralisé (5 rôles), vérifications côté serveur sur chaque route admin
Cryptographie (CRYP)	Conforme	Secrets chiffrés au repos AES-256-GCM ; pas de secret en clair en base
Validation des entrées (INPV)	Partiel	zod sur routes sensibles ; couverture à généraliser ; ORM = anti-injection SQL
Configuration (CONF)	Conforme	En-têtes HSTS/CSP/X-Frame/nosniff ; CI bloque les secrets en dur
Anti-automatisation (BUSL)	Partiel	Rate-limit par e-mail (pas par IP), store en mémoire non distribué

2. Périmètre et méthodologie

L'audit suit la structure de l'**OWASP Web Security Testing Guide (WSTG) v4.2**, en mode *white-box* (revue de code source). Chaque catégorie WSTG est évaluée et reliée aux contrôles effectivement présents dans le code. Les constats sont notés par sévérité (Critique / Élevé / Moyen / Faible / Info) selon l'impact et l'exploitabilité.

Artefacts analysés : `src/lib/auth.ts`, `middleware.ts`, `lib/permissions.ts`, `lib/secret-crypto.ts`, `lib/rate-limit.ts`, `lib/login-lockout.ts`, `lib/logger.ts`, `next.config.js`, les 27 routes API et le schéma Prisma (20 modèles).

3. Résultats par catégorie WSTG

WSTG-CONF — Configuration & déploiement

En-têtes de sécurité globaux définis dans `next.config.js` : `Strict-Transport-Security`, `Content-Security-Policy` (avec `frame-ancestors 'none'`), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`. La CI (*Security & Quality*) échoue si un secret en dur ou une dépendance vulnérable (high/critical en prod) est détecté. Conforme

M-1 (Moyen) — CSP à durcir. Vérifier l'absence de `'unsafe-inline'` / `'unsafe-eval'` dans les directives `script-src` ; viser une CSP stricte avec nonces pour neutraliser le XSS résiduel.

WSTG-ATHN — Authentification

- Mots de passe hachés via **bcrypt**, **coût 12** ; comparaison à temps constant.
- Anti-bruteforce** : 10 tentatives / e-mail / 15 min, plus **verrouillage de compte** configurable (seuil + durée).
- Politique de mot de passe** administrable : longueur, complexité, expiration, changement forcé à la première connexion.
- Réinitialisation par l'admin avec mot de passe temporaire (envoi e-mail si SMTP configuré).

M-2 (Moyen) — Limitation par e-mail uniquement. Le compteur anti-bruteforce est indexé sur l'e-mail (documenté `F003b`) : un attaquant peut faire du *credential stuffing* multi-comptes depuis une IP unique. **Reco** : ajouter une clé de limitation par IP et, en multi-instance, un store partagé (Redis).

WSTG-SESS — Gestion de session

Sessions **JWT** (next-auth, stratégie `jwt`), durée réduite à **8 heures** (vs 30 j par défaut). Cookie de session : `httpOnly: true`, `sameSite: 'lax'`, `secure` activé en HTTPS (`useSecureCookies` aligné sur le schéma d'URL). Sessions invalidées pour les comptes suspendus (refus côté callback). **Conforme**

WSTG-ATHZ — Autorisation

Contrôle d'accès centralisé dans `lib/permissions.ts` (matrice documentée, 5 rôles : `LECTEUR` · `ANALYSTE` · `RISK_MANAGER` · `RSSI` · `ADMIN` + grant `EDITION` par analyse). Le `middleware.ts` (`withAuth`) protège toutes les routes hors statiques. Chaque route d'administration revérifie le rôle **côté serveur** (`requireAdmin`) — pas de confiance au client. **Conforme**

F-1 (Faible) — IDOR. Confirmer systématiquement, sur les routes d'analyse, l'appartenance/partage de la ressource à l'utilisateur (pas seulement le rôle global) pour prévenir tout accès direct à l'objet (Insecure Direct Object Reference).

WSTG-CRYP — Cryptographie

Secrets sensibles **chiffrés au repos en AES-256-GCM** (`lib/secret-crypto.ts`) : mot de passe SMTP, *Client Secret* OIDC, secret du fournisseur SMS. Vérifié : la base stocke un blob `enc:v1:...`, jamais la valeur en clair. Codes MFA stockés sous forme de HMAC-SHA256, vérification à temps constant. **Conforme**

WSTG-INPV — Validation des entrées

Validation par schémas **zod** sur les routes sensibles (config org, SSO, SMTP, politique de mot de passe...). L'ORM **Prisma** élimine l'injection SQL (requêtes paramétrées). Protection anti-injection CSV sur les exports (testée, `spreadsheet-safe`). **Partiel**

M-3 (Moyen) — Généraliser la validation. zod n'est présent que sur 8 routes ; étendre la validation systématique (taille, type, bornes) à toutes les routes acceptant un corps utilisateur, et neutraliser le HTML/Markdown rendu issu de saisies libres.

WSTG-BUSL — Logique métier & anti-automatisation

M-4 (Moyen) — Store de limitation en mémoire. Les compteurs anti-bruteforce/anti-abus sont en mémoire processus : remis à zéro à chaque redéploiement et non partagés entre instances. Acceptable en mono-instance ; prévoir Redis pour un déploiement scalé.

WSTG-ERRH — Gestion des erreurs & journalisation

Piste d'audit complète (`lib/logger.ts`) : connexions, échecs, verrouillages, changements de rôle, modifications de configuration, exports — avec capture d'IP. Les secrets sont **rédigés** (`[REDACTED]`) dans les journaux (testé : `audit-redact`). **Conforme**

WSTG-IDNT — Gestion des identités

Provisionnement maîtrisé (inscription + création par admin), désactivation automatique des comptes inactifs (`account-lifecycle` , testé), suspension/réactivation, cycle de vie du mot de passe. **Conforme**

4. Point de vigilance — MFA

F-2 (Faible, traité) — MFA non appliqué. Le MFA (e-mail/SMS) disposait d'une interface de configuration sans application réelle à la connexion. Pour la v1, **l'interface a été masquée** (flag `MFA_UI_VISIBLE=false`) afin d'éviter tout faux sentiment de sécurité. Les fondations techniques (génération/vérification OTP, envoi SMS Twilio, modèle `MfaChallenge`) sont en place ; l'intégration au flux `authorize()` est planifiée pour une version ultérieure.

5. Tableau de synthèse des constats

Réf.	Sévérité	Catégorie	Constat	Recommandation
------	----------	-----------	---------	----------------

M-1	Moyen	CONF	CSP à durcir (inline/eval)	CSP stricte + nonces
M-2	Moyen	ATHN/BUSL	Rate-limit par e-mail seul	Ajouter clé par IP
M-3	Moyen	INPV	Validation non généralisée	zod sur toutes les routes
M-4	Moyen	BUSL	Store rate-limit en mémoire	Redis en multi-instance
F-1	Faible	ATHZ	Vérifier l'IDOR par ressource	Contrôle d'appartenance
F-2	Faible	ATHN	MFA non appliqué (traité v1)	Intégrer à authorize()
I-1	Info	CONF	3 CVE high en devDeps (build)	Non expédié ; suivi MAJ vite/esbuild

6. Conclusion

Verdict : publiable en v1. ACRA implémente correctement les contrôles de sécurité essentiels attendus d'une application manipulant des données de risque cyber. Aucun blocage critique. Les recommandations Moyennes relèvent du **durcissement de défense en profondeur** et peuvent être planifiées post-v1, avec en priorité la limitation par IP et la généralisation de la validation des entrées avant une exposition publique à grande échelle.

Audit white-box réalisé le 14/06/2026 sur la branche `main`. Méthodologie OWASP WSTG v4.2. Ce rapport reflète l'état du code à la date d'analyse.